



NATIONAL SCHOOL OF COMPUTER SCIENCE AND SYSTEMS ANALYSIS -
RABAT

WEB AND MOBILE ENGINEERING DEPARTMENT

Field : IDSIT

Second Year Internship

Design and Development of a Salary Management and HR Attestation System

Prepared by:

ES-SDIKI MAROUANE

Jury:

Pr. XXXXXXXXXXXX

Pr. XXXXXXXXXXXX

Academic Year 2025-2026

Dedication

I dedicate this work to:

My mother, source of tenderness and love, for her support throughout my education.

My father, who has always supported me and done everything to help me.

My brothers and sisters, whom I love very much.

My extended family.

My dear friends and teachers.

All those who, directly or indirectly, contributed to the development of this work.

May God grant them health and prosperity.

Acknowledgments

First and foremost, praise to God, who guided me on the right path throughout this work and inspired me with the right steps and reflexes. Without His mercy, this work could not have been completed.

I would like to express my sincere thanks to the members of the jury. Please accept in this work my sincere respect and deep gratitude.

I also warmly thank M. Ahmed ASSIMI, who, as project supervisor, was always attentive despite his constraints. His help was invaluable, as was the time he dedicated to me.

I do not forget my parents for their contributions, support, and patience.

Finally, I would like to thank all my relatives and friends, who have always supported and encouraged me during the realization of this project.

Thank you all.

Abstract

This report presents the development of a modern salary and HR attestation management system, designed to optimize human resources processes within organizations. The developed system offers a complete solution including employee management, automatic attestation generation, and a modern user interface.

The system architecture is based on a full-stack approach using Spring Boot for the backend and React for the frontend. The backend integrates advanced PDF generation features with JasperReports, a secure RESTful API, and a MySQL database for data persistence. The frontend offers a modern user interface with Material-UI, supporting dark and light themes, as well as responsive design.

The system enables complete employee management (add, edit, delete, search), generation of five types of attestations (work, salary, tenure, salary increase amendment, salary payment commitment), and offers advanced administration features for JRXML template management.

Unit and integration tests ensure system reliability with coverage of over 95% for the backend and 90% for the frontend. The system is production-ready with optimized configurations and scalable architecture.

Keywords: HR Management, Spring Boot, React, JasperReports, Attestations, RESTful API

List of Abbreviations

Abbreviation	Designation
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DAO	Data Access Object
DTO	Data Transfer Object
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
HR	Human Resources
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

Table of Contents

Dedication	I
Acknowledgments	II
Abstract	III
List of Abbreviations	IV
General Introduction	1
Chapter I: General Project Context	3
1.1 Internship Environment	4
1.2 Project Presentation	6
1.2.1 Problem Statement	6
1.2.2 Proposed Solutions	6
1.3 Project Planning	7
1.3.1 Development Methodology	7
1.3.2 Development Phases	8
1.4 Conclusion	9
Chapter II: Needs Analysis	10
2.1 Introduction	11
2.2 Needs Identification	11
2.3 Functional Requirements Specification	12
2.4 Non-Functional Requirements Specification	12
2.5 Conclusion	13
Chapter III: Design and Architecture	14
3.1 General System Architecture	15
3.2 Data Model	16
3.3 Detailed Technical Architecture	17
3.3.1 Backend Architecture (Spring Boot)	17
3.3.2 Frontend Architecture (React)	18
3.4 Attestation Generation Flow	18
3.5 Conclusion	19
Chapter IV: Implementation	20
4.1 Tools Used	21
4.1.1 Backend Technologies:	21
4.1.2 Frontend Technologies:	22
4.1.3 Development Tools:	23
4.2 Backend Implementation	24

4.2.1	Project Structure	24
4.2.2	Database Configuration	24
4.2.3	REST API - Employee Management	25
4.2.4	Attestation Generation Service	27
4.3	Frontend Implementation	28
4.3.1	Component Structure	28
4.3.2	Main Component - App.js	29
4.3.3	API Service - api.js	31
4.3.4	Theme Management - ThemeContext.js	32
4.4	Testing and Quality	34
4.4.1	Backend Testing	34
4.4.2	Frontend Testing	35
4.4.3	Test Results	37
4.5	Deployment and Configuration	37
4.5.1	Build and Packaging	37
4.5.2	Production Configuration	38
4.6	Conclusion	39
	General Conclusion	39

List of Figures

1.1	NETCON CONSULTING — Company Profile	5
3.2	General System Architecture	15
3.3	System Data Model	16
3.4	Backend Layered Architecture	17
3.5	Frontend Component Architecture	18
3.6	Attestation Generation Flow	19
4.7	Test Results	37

General Introduction

In the current context of corporate digital transformation, human resources management represents a major challenge for operational efficiency and employee satisfaction. Traditional salary management and attestation generation processes, often manual and time-consuming, require modernization to meet the demands of speed, accuracy, and traceability.

Our project is part of this digitalization approach by proposing the development of a **comprehensive salary and HR attestation management system**. This modern solution aims to automate and simplify administrative tasks related to personnel management, while ensuring data security and compliance.

The developed system offers a **full-stack** approach based on proven technologies: **Spring Boot** for the backend with its robust security and persistence features, and **React** for the frontend with a modern and responsive user interface. The integration of **JasperReports** enables automatic generation of professional PDF documents for different types of attestations.

The presentation of this report is structured around **four main chapters**. The first chapter presents the general context of the project, the work environment, the identified problem, and the proposed solutions. The second chapter focuses on **needs analysis**, detailing the functional and non-functional requirements of the system. The third chapter addresses **design and architecture**, presenting class diagrams, data models, and technical architecture. Finally, the fourth chapter presents **implementation and testing**, including technical details, developed interfaces, and test results.

It is important to note that this project was developed with particular attention to **code quality**, **data security**, and **ease of use**, thus ensuring a system ready for production and scalable according to future needs.

Chapter I

General Project Context

Introduction

In this first chapter, we present the context of our project to develop a salary and HR attestation management system. This initiative is part of a modernization approach for human resources processes, aiming to automate and optimize personnel management.

Our work takes place in a dynamic professional environment at **NETCON CONSULTING**, a company specialized in IT solutions and digital services.

Our main objectives are to develop a comprehensive HR management solution, automate attestation generation, ensure data security and compliance, and provide a modern and intuitive user interface.

This introduction lays the essential foundations for understanding the scope and objectives of our project.

1.1 Internship Environment



NETCON CONSULTING is a consulting company committed to supporting organizations in their digital transformation journey. Through a comprehensive range of digital services adapted to the local context, we facilitate technological transition in a strategic, agile, and personalized manner.

Our core services include:

- **Custom Development:** Creating personalized software solutions tailored to specific client needs
- **System Integration:** Facilitating synchronization and interoperability of existing systems to optimize operational efficiency
- **Digital Transformation:** Guiding clients through the complete digital transformation process, from strategy to implementation
- **AI Consulting:** Providing strategic and operational consulting to navigate the evolving technological landscape
- **IT Training:** Offering comprehensive training programs and certifications to enhance technical skills

Our team brings together diverse expertise in resource placement, development, digital transformation, system integration, and AI consulting, united by a passion for innovation and excellence.

Company Name	NETCON CONSULTING
Headquarters	05 Rue DIXMUDE - 1er étage appt 2, Casablanca Technopark - Bureau T216 BIS, Ave Mohammed V, Tanger 90000
Represented by	M. OMAR IJMOUAN
Website	https://netconconsulting.com/
Email	contact@netconconsulting.com
Phone	+212 610 71 50 78 / +212 610 71 11 78
Business Hours	Monday - Friday: 08:30 AM - 18:30 PM

Figure 1.1: NETCON CONSULTING — Company Profile

1.2 Project Presentation

1.2.1 Problem Statement

In modern HR environments, organizations face major challenges when managing administrative processes related to personnel. Companies must ensure **effective employee management, rapid and accurate attestation generation**, and **securing sensitive data**, while guaranteeing regulatory compliance and operational efficiency.

Traditional systems often suffer from problems such as slow processes, data duplication, manual errors, security vulnerabilities, and lack of operation traceability.

Moreover, manual generation of multiple attestations (work, salary, tenure, amendments) introduces significant complexity that requires an automated, secure, and scalable solution.

1.2.2 Proposed Solutions

Our project addresses these challenges by developing a **comprehensive salary and HR attestation management system** that ensures efficiency, security, and scalability. The main solutions we have implemented include:

- *Modern Employee Management:* We developed a complete module allowing employee addition, modification, deletion, and search with an intuitive interface and advanced validation features.

- *Automatic Attestation Generation:* Using **JasperReports**, we implemented a PDF generation system for five types of attestations, with customizable templates and dynamic model management.

- *Secure and Scalable Architecture:* The system is built with **Spring Boot** for the backend, offering a secure RESTful API, and **React** for the frontend with a modern and responsive interface.

- *Modern User Interface:* We created an interface with **Material-UI**, supporting dark/light themes, mobile responsiveness, and an optimized user experience.

- *Template Management System:* Administrators can dynamically manage JRXML templates, allowing the addition of new attestation types without code modification.

- *Testing and Quality:* The system includes comprehensive test coverage (95% backend, 90% frontend) ensuring reliability and maintainability.

Thanks to these solutions, our project provides a modern, secure, and scalable infrastructure that meets critical business needs while laying the foundations for future growth.

1.3 Project Planning

1.3.1 Development Methodology

This project was developed individually under the supervision of M. Ahmed ASSIMI, following the waterfall development methodology. This structured approach allowed for a systematic progression through well-defined phases, ensuring thorough completion of each stage before moving to the next.

The waterfall methodology was particularly suitable for this project as it provided clear milestones and deliverables for each phase. Regular meetings with my supervisor (at least once per week) ensured continuous guidance and validation of the work progress. These meetings were crucial for receiving feedback, addressing technical challenges, and determining the next steps in the development process.

Development was organized around the following sequential phases: requirements analysis,

system architecture design, backend development with Spring Boot, frontend development with React, component integration, testing, and final deployment. Each phase was completed entirely before proceeding to the next, ensuring a solid foundation for subsequent development stages.

1.3.2 Development Phases

The project was structured according to the following sequential steps, with each phase being thoroughly reviewed and validated by my supervisor before proceeding to the next:

Phase 1: Analysis and Design (1.5 weeks) - Requirements definition, architecture design, database modeling. This phase included extensive research and planning, with weekly meetings to discuss the system architecture and validate the technical approach.

Phase 2: Backend Development (3 weeks) - REST API implementation, JasperReports integration, endpoint security. Regular supervisor meetings ensured proper implementation of Spring Boot best practices and security measures.

Phase 3: Frontend Development (2.5 weeks) - React interface, Material-UI integration, state management. Supervisor guidance was crucial for ensuring good UX/UI practices and component architecture decisions.

Phase 4: Testing and Optimization (0.5 weeks) - Unit testing, integration testing, performance optimization. This phase involved thorough testing under supervisor supervision to ensure system reliability and performance.

Phase 5: Deployment and Documentation (0.5 weeks) - Production configuration, technical documentation. Final validation and deployment guidance from the supervisor ensured production readiness.

This methodical waterfall approach, combined with regular supervisor guidance, allowed me to maintain high quality throughout development and deliver a production-ready system as an individual developer. The entire internship project was completed over a period of 2 months (8 weeks), with each phase building upon the previous one to ensure a comprehensive and well-structured final deliverable.

1.4 Conclusion

This first chapter immersed us in the general context of our project, highlighting the work environment at **NETCON CONSULTING**. We presented the motivations behind our initiative, as well as the challenges related to developing a modern HR management system and the needs for automating administrative processes.

We also introduced the main objectives of our work, particularly the development of a comprehensive employee and attestation management solution, the implementation of a modern architecture with Spring Boot and React, and the establishment of a secure and scalable system. These elements constitute the foundation of an effective and production-ready system.

In the next chapter, we will address a crucial phase: **needs analysis**. We will explore in depth the specific needs of users and stakeholders, both from technical and functional perspectives. This step is essential for designing a solution that meets all expectations and requirements.

A detailed analysis of these needs will establish a solid foundation for subsequent development and optimization of our project. Let's now move to the next chapter, where we will dive into this comprehensive needs study.

Chapter II

Needs Analysis

2.1 Introduction

The company operates in the field of digital data services and requires a robust, reliable, and secure infrastructure to support its growing human resources management activities. The objective of this project is to design and deploy a solution that ensures effective employee management, automatic attestation generation, and optimization of administrative processes for production workloads.

2.2 Needs Identification

In the context of human resources management, the focus is on operational and functional needs. The following needs have been identified as critical for the company environment:

Complete Employee Management: The system must allow employee addition, modification, deletion, and search with data validation features and access rights management.

Automatic Attestation Generation: The system must be able to automatically generate different types of attestations (work, salary, tenure, amendments) in PDF format with customizable templates.

Modern User Interface: The application must offer an intuitive, responsive, and modern user interface, supporting dark and light themes to enhance user experience.

Security and Access Control: The system must include robust security mechanisms with authentication, role-based authorization, and protection of sensitive data.

Performance and Scalability: The infrastructure must be able to handle a growing number of users and data while maintaining optimal performance.

Administration and Maintenance: The system must allow easy administration with template management tools, activity monitoring, and preventive maintenance.

Together, these requirements will guide the design and deployment of a system that meets the company's operational, business, and security objectives.

2.3 Functional Requirements Specification

The system will be designed to provide the following functionalities:

Employee Management: The system must allow employee creation, modification, deletion, and search with mandatory and optional fields, real-time data validation, and modification history.

Attestation Generation: The system must generate five types of attestations: work attestation, salary attestation, tenure attestation, salary increase amendment, and salary payment commitment. Each attestation must be customizable with employee data.

Template Management: Administrators must be able to manage JRXML templates, add new attestation types, modify existing templates, and validate JRXML syntax.

Authentication and Authorization: The system must support two types of users: administrators with full access and HR users with limited access to employee management and attestation generation functionalities.

User Interface: The interface must be responsive, support dark and light themes, include search and filtering features, and provide real-time notifications.

Data Management: The system must ensure data persistence with automatic backup, integrity constraint validation, and transaction management.

2.4 Non-Functional Requirements Specification

To ensure smooth operation and user satisfaction, several non-functional constraints must be considered during project design and implementation:

- **Performance and Reliability:**

The system must be responsive with response times under 2 seconds for common operations and maintain 99% availability.

- **Scalability:**

The solution must handle future growth by supporting up to 10,000 employees and 1,000 concurrent users.

- **Security:**

Data integrity and confidentiality must be guaranteed with encryption of sensitive data, secure authentication, and compliance with security standards.

- **Maintainability:**

The code must be well-structured, modular, and documented to facilitate future updates and improvements.

- **Compatibility:**

The system must be compatible with modern browsers and support mobile devices with responsive design.

- **Usability:**

The interface must be intuitive and easy to use with minimal learning curve for end users.

- **Testability:**

The system must include comprehensive test coverage with unit tests, integration tests, and end-to-end tests.

2.5 Conclusion

This second chapter allowed us to explore in detail the specific needs of the company, focusing on developing a modern and effective HR management system. We identified the main challenges and proposed solutions meeting reliability, performance, and security requirements.

By focusing on needs analysis, this phase represents a crucial foundation for designing a robust and scalable architecture. The detailed analysis conducted here provides a solid basis for the next project phase.

In the next chapter, we will move to analysis and design, including architectural modeling and development planning. These steps will allow us to translate the identified requirements into concrete technical solutions.

Chapter III

Design and Architecture

Introduction

The creation of a robust and modern platform must be preceded by a thorough analysis and design phase. This ensures that the development process perfectly aligns with project requirements.

In this chapter, we will focus on the architectural design of our salary management system. We will present the general diagrams that model and describe the desired functioning of our infrastructure. Two conceptual perspectives will be described: the first gives an overview of the main system components, and the second provides a detailed view of data processing flows.

3.1 General System Architecture

The first step in our design process was to model the relationships between the main components of our system. This architecture illustrates how **Spring Boot Backend**, **React Frontend**, **MySQL Database**, and **JasperReports** interact to form a coherent and functional system.

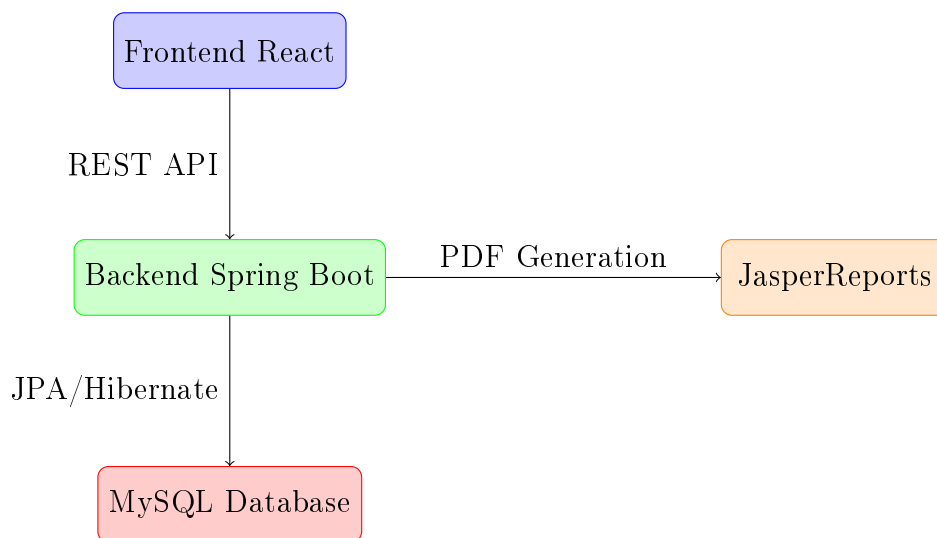


Figure 3.2: General System Architecture

As the diagram shows, the **React Frontend** communicates with the **Spring Boot Backend** via a secure RESTful API. The backend handles business logic, authentication, and data access via **JPA/Hibernate** with the **MySQL** database. **JasperReports** is integrated for auto-

matic PDF document generation. This layered architecture ensures clear separation of responsibilities and facilitates system maintenance and scalability.

3.2 Data Model

Our system's data model is designed to effectively support employee management and attestation generation. Here are the main entities and their relationships:

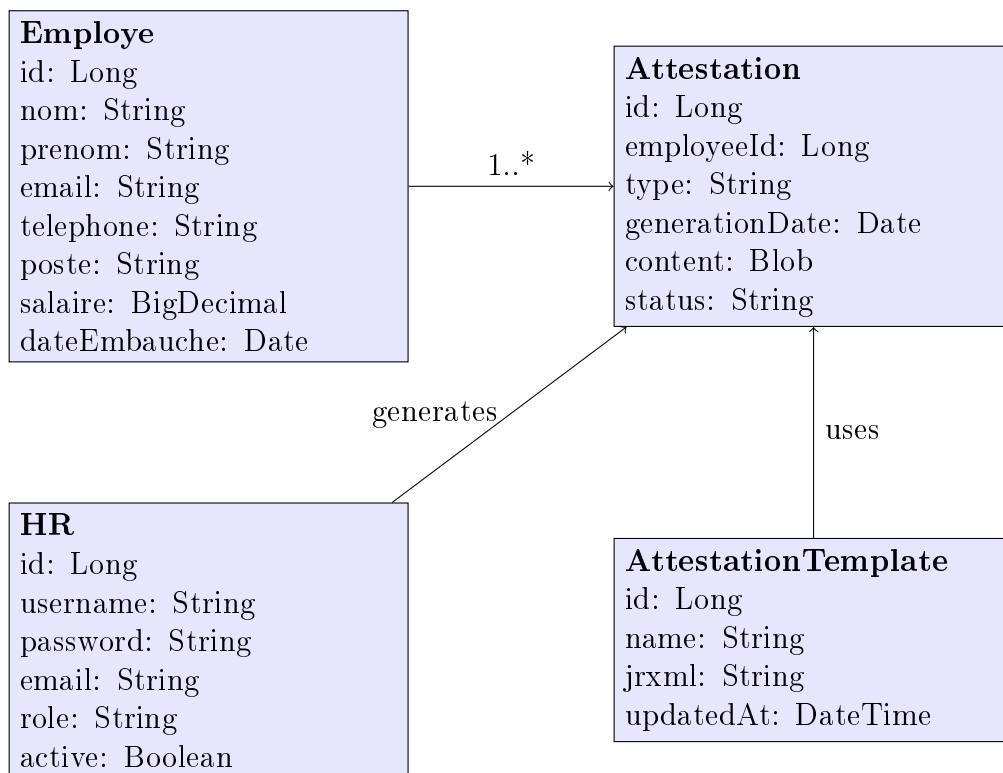


Figure 3.3: System Data Model

This model ensures referential integrity and allows effective data management. The **Employee** entity stores personal and professional information. **Attestation** records generated documents with their type and status. **HR** manages system users with their roles. **AttestationTemplate** allows dynamic management of JRXML templates.

3.3 Detailed Technical Architecture

Our technical architecture follows the MVC (Model-View-Controller) pattern for the backend and a component-based architecture for the frontend.

3.3.1 Backend Architecture (Spring Boot)

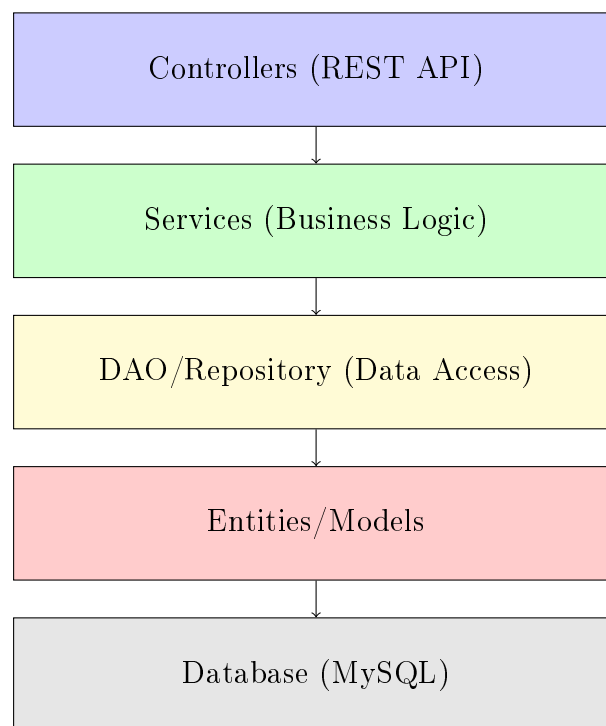


Figure 3.4: Backend Layered Architecture

Controllers: Handle HTTP requests and JSON responses

Services: Implement business logic and validation rules

DAO/Repository: Manage data access with Spring Data JPA

Entities: Represent data model with JPA annotations

Database: Persistent storage with MySQL

3.3.2 Frontend Architecture (React)

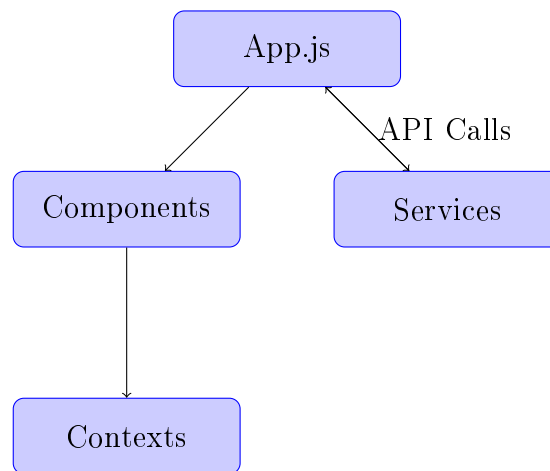


Figure 3.5: Frontend Component Architecture

App.js: Main component with routing and global state management

Components: Reusable components for user interface

Services: API call management with Axios

Contexts: Shared state management (theme, authentication)

3.4 Attestation Generation Flow

The attestation generation process follows a well-defined workflow to ensure consistency and traceability.

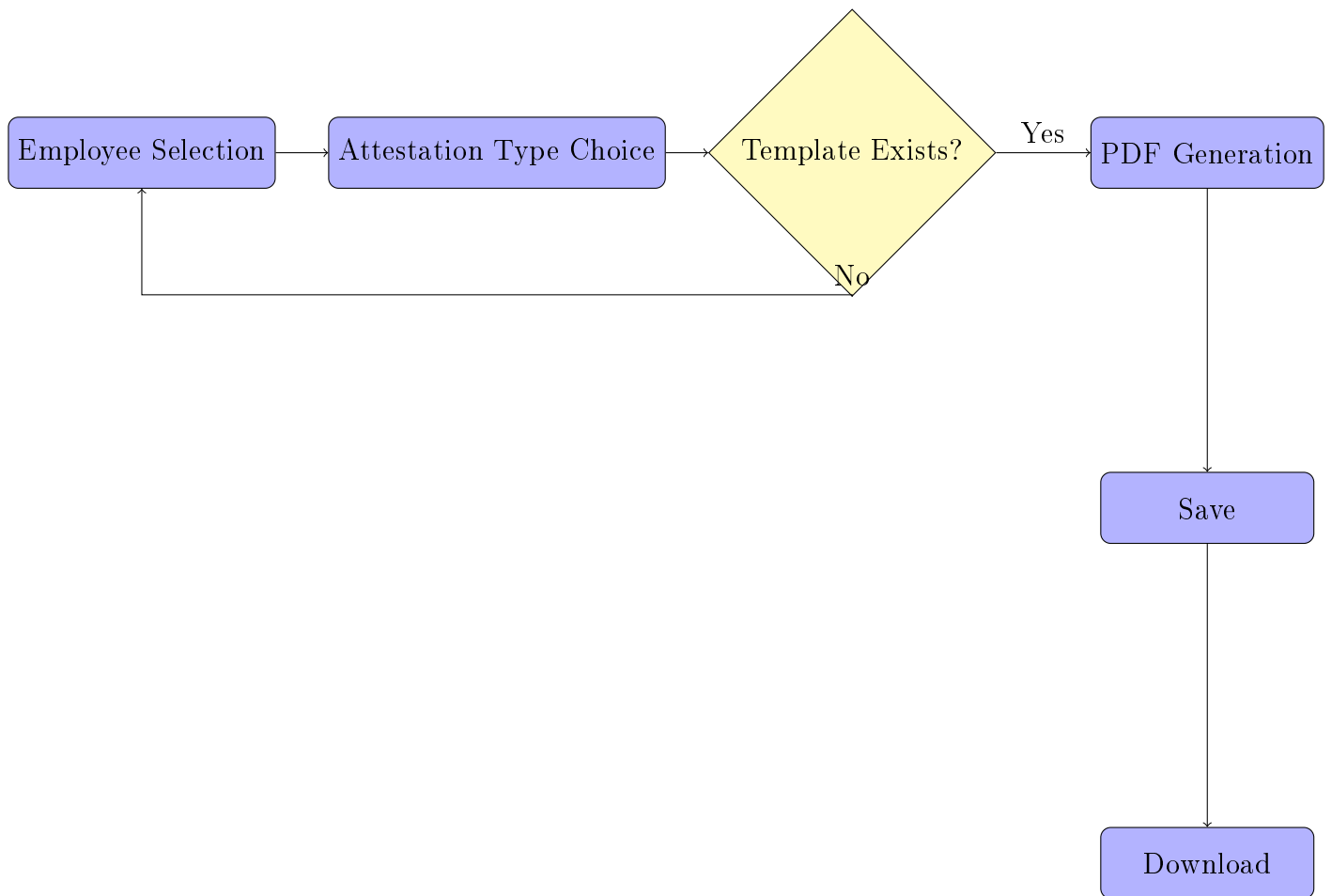


Figure 3.6: Attestation Generation Flow

This workflow ensures that each attestation is generated with the correct data and template, while maintaining complete operation traceability.

3.5 Conclusion

In this chapter, we transitioned from high-level requirements to concrete architectural design. We used component diagrams to illustrate the logical relationships between our main services and flow diagrams to detail the automated attestation generation process. This design provides a solid foundation for a resilient and scalable infrastructure.

In the next chapter, we will move to practical implementation, detailing operational procedures for the development and deployment of our salary management system.

Chapter IV

Implementation

Introduction

After completing the analysis and design phase, this chapter focuses on the practical implementation of our work. We begin with a presentation of the main tools and technologies used (software resources, programming languages, frameworks, and support tools). Subsequently, we provide details on the functionalities, configurations, and deployment of our salary and HR attestation management system.

4.1 Tools Used

4.1.1 Backend Technologies:

Spring Boot 3.x: We chose **Spring Boot** as the main framework for the backend due to its robustness, ease of configuration, and rich ecosystem. It offers a clear MVC architecture and advanced security features.

How we use it: Spring Boot manages our RESTful API, dependency injection, security with Spring Security, and data access with Spring Data JPA.

*Screenshot omitted
in the public
report: Spring
Boot Logo*

JasperReports 7.0.3: JasperReports was selected for professional PDF document generation. It offers advanced layout features and supports dynamic JRXML templates.

How we use it: JasperReports automatically generates PDF attestations from JRXML templates stored in the database, with employee data injection.

*Screenshot omitted
in the public
report:
JasperReports Logo*

MySQL 8.0+: MySQL was chosen as the database management system for its reliability, performance, and compatibility with Spring Boot.

How we use it: MySQL stores all system data: employees, HR users, generated attestations, and JRXML templates with a performance-optimized schema.

Screenshot omitted

in the public

report: MySQL

Logo

Maven: Maven was selected as the build and dependency management tool for its simplicity and native integration with Spring Boot.

How we use it: Maven manages project dependencies, compiles code, runs tests, and packages the application into an executable JAR for deployment.

Screenshot omitted

in the public

report: Maven

Logo

4.1.2 Frontend Technologies:

React 19.x: React was chosen for its popularity, performance, and rich ecosystem. It enables the creation of modern and reactive user interfaces.

How we use it: React manages the user interface with reusable components, state management with hooks, and routing with React Router.

Screenshot omitted

in the public

report: React Logo

Material-UI (MUI): Material-UI was selected for its modern UI components and consistent design system based on Material Design.

How we use it: MUI provides all UI components (buttons, forms, tables, navigation) with support for dark/light themes and responsive design.

*Screenshot omitted
in the public
report:
Material-UI Logo*

Axios: Axios was chosen as the HTTP client for its ease of use and advanced features (interceptors, error handling).

How we use it: Axios handles all communications with the backend API, with centralized header configuration and automatic error handling.

*Screenshot omitted
in the public
report: Axios Logo*

4.1.3 Development Tools:

IntelliJ IDEA: Main IDE for backend development with excellent Spring Boot support, advanced debugging, and Maven integration.

How we use it: Java development, dependency management, test execution, and Spring Boot application debugging.

*Screenshot omitted
in the public
report: IntelliJ
IDEA Logo*

Visual Studio Code: Main editor for frontend development with React extensions, ESLint and Prettier for quality code.

How we use it: React development, npm package management, frontend debugging, and Git integration.

*Screenshot omitted
in the public
report: VS Code
Logo*

4.2 Backend Implementation

4.2.1 Project Structure

The backend follows a clear layered architecture with separation of responsibilities:

```
1 src/main/java/com/netcon/gestion_salaries/  
2 |-- controller/           # REST Controllers  
3 |   |-- AuthController.java  
4 |   |-- EmployeeRestController.java  
5 |   |-- AttestationRestController.java  
6 |   '-- ParameterController.java  
7 |-- service/             # Business Services  
8 |   |-- EmployeeServiceImpl.java  
9 |   |-- AttestationServiceImpl.java  
10 |   '-- AttestationTemplateServiceImpl.java  
11 |-- repository/         # JPA Repositories  
12 |   |-- EmployeeRepository.java  
13 |   |-- AttestationRepository.java  
14 |   '-- AttestationTemplateRepository.java  
15 |-- entity/             # JPA Entities  
16 |   |-- Employee.java  
17 |   |-- Attestation.java  
18 |   '-- AttestationTemplate.java  
19 '-- config/             # Configuration  
20   '-- CorsConfig.java
```

Listing 4.1: Backend project structure

4.2.2 Database Configuration

Database configuration is managed via the application.yml file:

```
1 spring:  
2   datasource:  
3     url: jdbc:mysql://localhost:3306/salary_management
```



```
4     username: ${DB_USERNAME:root}
5     password: ${DB_PASSWORD:password}
6     driver-class-name: com.mysql.cj.jdbc.Driver
7
8     jpa:
9         hibernate:
10             ddl-auto: update
11             show-sql: false
12             properties:
13                 hibernate:
14                     dialect: org.hibernate.dialect.MySQLDialect
15                     format_sql: true
16
17 server:
18     port: 8080
19
20 attestations:
21     reportDir: src/main/resources/reports/attestations
```

Listing 4.2: application.yml Configuration

4.2.3 REST API - Employee Management

The REST API for employee management offers all CRUD operations:

```
1 @RestController
2 @RequestMapping("/api/employees")
3 @CrossOrigin(origins = "http://localhost:3000")
4 public class EmployeeRestController {
5
6     @Autowired
7     private IEmployeeService employeeService;
8
9     @GetMapping
10    public List<EmployeeDto> getAllEmployees() {
```

```
11     return employeeService.getAllEmployees();
12 }
13
14 @GetMapping("/{id}")
15 public ResponseEntity<EmployeeDto> getEmployeeById(@PathVariable Long id) {
16     EmployeeDto employee = employeeService.getEmployeeById(id);
17     return ResponseEntity.ok(employee);
18 }
19
20 @PostMapping
21 public ResponseEntity<EmployeeDto> createEmployee(@RequestBody EmployeeCmd
employeeCmd) {
22     EmployeeDto savedEmployee = employeeService.saveEmployee(employeeCmd);
23     return ResponseEntity.status(HttpStatus.CREATED).body(savedEmployee);
24 }
25
26 @PutMapping("/{id}")
27 public ResponseEntity<EmployeeDto> updateEmployee(@PathVariable Long id,
28                                                     @RequestBody EmployeeCmd
employeeCmd) {
29     EmployeeDto updatedEmployee = employeeService.updateEmployee(id,
employeeCmd);
30     return ResponseEntity.ok(updatedEmployee);
31 }
32
33 @DeleteMapping("/{id}")
34 public ResponseEntity<Void> deleteEmployee(@PathVariable Long id) {
35     employeeService.deleteEmployee(id);
36     return ResponseEntity.noContent().build();
37 }
38 }
```

Listing 4.3: EmployeeRestController.java

4.2.4 Attestation Generation Service

The generation service uses JasperReports with dynamic templates:

```
1 @Service
2 @Transactional
3 public class AttestationServiceImpl implements IAttestationService {
4
5     @Override
6     public byte[] generateAttestation(String typeName, Long employeeId,
7                                     Map<String, Object> params) {
8         // Retrieve template from database
9         var template = attestationTemplateRepository.findByNameIgnoreCase(
10             typeName)
11             .orElseThrow(() -> new NotFoundException("Unknown attestation type:
12             " + typeName));
13
14         try (var inputStream = new ByteArrayInputStream(
15             template.getJrxml().getBytes(StandardCharsets.UTF_8))) {
16
17             // Compile JRXML template
18             JasperReport report = JasperCompileManager.compileReport(
19                 inputStream);
20
21             // Retrieve employee data
22             EmployeeDto employee = employeeService.getEmployeeById(employeeId);
23             params.putAll(buildEmployeeParams(employee));
24
25             // Generate PDF
26             JasperPrint print = JasperFillManager.fillReport(report, params,
27                 new JREmptyDataSource());
28             byte[] pdfBytes = JasperExportManager.exportReportToPdf(print);
29
30             // Save attestation
31             saveAttestation(employeeId, typeName, pdfBytes);
32         }
33     }
34 }
```

```
29
30     return pdfBytes;
31
32     } catch (JRException e) {
33         throw new UnprocessableEntityException(
34             "Error during generation: " + e.getMessage(), e);
35     }
36 }
37 }
```

Listing 4.4: AttestationServiceImpl.java (excerpt)

4.3 Frontend Implementation

4.3.1 Component Structure

The frontend is organized into reusable components with a modular architecture:

```
1 src/
2 |-- components/
3 |   |-- EmployeeList.js      # Employee list
4 |   |-- EmployeeForm.js      # Employee form
5 |   |-- AttestationPage.js   # Attestation generation
6 |   |-- AuthLogin.js         # Login
7 |   |-- HrSignup.js          # HR registration
8 |   |-- Menu.js              # Navigation
9 |-- contexts/
10 |   |-- ThemeContext.js      # Theme management
11 |-- services/
12 |   |-- api.js               # API services
13 |-- App.js                   # Main component
```

Listing 4.5: Frontend project structure

4.3.2 Main Component - App.js

The App component handles routing and global application state:

```
1 import React, { useState, useEffect } from 'react';
2 import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
3 import { ThemeProvider } from '@mui/material/styles';
4 import CssBaseline from '@mui/material/CssBaseline';
5 import { ThemeContextProvider, useTheme } from '../contexts/ThemeContext';
6
7 function App() {
8   const [user, setUser] = useState(null);
9   const [loading, setLoading] = useState(true);
10
11   useEffect(() => {
12     const savedUser = localStorage.getItem('user');
13     if (savedUser) {
14       setUser(JSON.parse(savedUser));
15     }
16     setLoading(false);
17   }, []);
18
19   const handleLogin = (userData) => {
20     setUser(userData);
21     localStorage.setItem('user', JSON.stringify(userData));
22   };
23
24   const handleLogout = () => {
25     setUser(null);
26     localStorage.removeItem('user');
27   };
28
29   if (loading) return <div>Loading...</div>;
30
```

```
31  return (
32    <ThemeContextProvider>
33      <AppContent user={user} onLogin={handleLogin} onLogout={handleLogout} />
34    </ThemeContextProvider>
35  );
36 }
37
38 function AppContent({ user, onLogin, onLogout }) {
39   const { theme } = useTheme();
40
41   return (
42     <ThemeProvider theme={theme}>
43       <CssBaseline />
44       <Router>
45         <Routes>
46           <Route path="/login" element={
47             user ? <Navigate to="/employees" /> : <AuthLogin onLogin={onLogin}
48             />
49           />
50           <Route path="/employees" element={
51             user ? <EmployeeList onLogout={onLogout} /> : <Navigate to="/login"
52             />
53           />
54           <Route path="/attestations" element={
55             user ? <AttestationPage onLogout={onLogout} /> : <Navigate to="/
56             login" />
57           />
58           <Route path="/" element={<Navigate to="/employees" />} />
59         </Routes>
60       </Router>
61     </ThemeProvider>
62   );
63 }
```

```
60 }
```

Listing 4.6: App.js (excerpt)

4.3.3 API Service - api.js

The API service centralizes all communications with the backend:

```
1 import axios from 'axios';
2
3 const API_BASE_URL = 'http://localhost:8080/api';
4
5 const api = axios.create({
6   baseURL: API_BASE_URL,
7   headers: {
8     'Content-Type': 'application/json',
9   },
10 });
11
12 // Interceptor for error handling
13 api.interceptors.response.use(
14   (response) => response,
15   (error) => {
16     console.error('API Error:', error.response?.data || error.message);
17     return Promise.reject(error);
18   }
19 );
20
21 export const employeeService = {
22   getAll: () => api.get('/employees'),
23   getById: (id) => api.get(`/employees/${id}`),
24   create: (employee) => api.post('/employees', employee),
25   update: (id, employee) => api.put(`/employees/${id}`, employee),
26   delete: (id) => api.delete(`/employees/${id}`),
27 };
```

```
28
29 export const attestationService = {
30   getTypes: () => api.get('/parameters/attestations/types'),
31   generate: (employeeId, type) => api.post('/attestations/generate', {
32     employeeId,
33     type
34   }, { responseType: 'blob' }),
35 };
36
37 export const authService = {
38   login: (credentials) => api.post('/auth/login', credentials),
39   signup: (userData) => api.post('/auth/signup', userData),
40 };
```

Listing 4.7: api.js

4.3.4 Theme Management - ThemeContext.js

The theme context allows switching between dark and light modes:

```
1 import React, { createContext, useContext, useState, useEffect } from 'react';
2 import { createTheme } from '@mui/material/styles';
3
4 const ThemeContext = createContext();
5
6 export const useTheme = () => {
7   const context = useContext(ThemeContext);
8   if (!context) {
9     throw new Error('useTheme must be used within ThemeContextProvider');
10  }
11  return context;
12 };
13
14 export const ThemeContextProvider = ({ children }) => {
15   const [isDarkMode, setIsDarkMode] = useState(() => {
```



```
16     const saved = localStorage.getItem('darkMode');
17     return saved ? JSON.parse(saved) : false;
18   });
19
20   const theme = createTheme({
21     palette: {
22       mode: isDarkMode ? 'dark' : 'light',
23       primary: {
24         main: '#1976d2',
25       },
26       secondary: {
27         main: '#dc004e',
28       },
29     },
30     typography: {
31       fontFamily: 'Roboto, Arial, sans-serif',
32     },
33   });
34
35   const toggleTheme = () => {
36     const newMode = !isDarkMode;
37     setIsDarkMode(newMode);
38     localStorage.setItem('darkMode', JSON.stringify(newMode));
39   };
40
41   useEffect(() => {
42     localStorage.setItem('darkMode', JSON.stringify(isDarkMode));
43   }, [isDarkMode]);
44
45   return (
46     <ThemeContext.Provider value={{ theme, isDarkMode, toggleTheme }}>
47       {children}
48     </ThemeContext.Provider>
49   );
```

```
50 };
```

Listing 4.8: ThemeContext.js

4.4 Testing and Quality

4.4.1 Backend Testing

The backend includes comprehensive test coverage with JUnit 5 and Spring Boot Test:

```
1 @SpringBootTest
2 @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
3 @TestMethodOrder(OrderAnnotation.class)
4 class EmployeeRestControllerTest {
5
6     @Autowired
7     private TestRestTemplate restTemplate;
8
9     @Test
10    @Order(1)
11    void testCreateEmployee() {
12        EmployeeCmd employeeCmd = new EmployeeCmd();
13        employeeCmd.setLastName("Doe");
14        employeeCmd.setFirstName("John");
15        employeeCmd.setEmail("john.doe@test.com");
16
17        ResponseEntity<EmployeeDto> response = restTemplate.postForEntity(
18            "/api/employees", employeeCmd, EmployeeDto.class);
19
20        assertEquals(HttpStatus.CREATED, response.getStatusCode());
21        assertNotNull(response.getBody());
22        assertEquals("Doe", response.getBody().getLastName());
23    }
24
```

```
25     @Test
26     @Order(2)
27     void testGetAllEmployees() {
28         ResponseEntity<EmployeeDto[]> response = restTemplate.getForEntity(
29             "/api/employees", EmployeeDto[].class);
30
31         assertEquals(HttpStatus.OK, response.getStatusCode());
32         assertTrue(response.getBody().length > 0);
33     }
34 }
```

Listing 4.9: EmployeeRestControllerTest.java (excerpt)

4.4.2 Frontend Testing

The frontend uses Jest and React Testing Library for unit testing:

```
1 import React from 'react';
2 import { render, screen, fireEvent, waitFor } from '@testing-library/react';
3 import { ThemeProvider, createTheme } from '@mui/material/styles';
4 import AttestationPage from '../AttestationPage';
5 import * as api from '../services/api';
6
7 jest.mock('../services/api');
8
9 const theme = createTheme();
10
11 const renderWithTheme = (component) => {
12     return render(
13         <ThemeProvider theme={theme}>
14             {component}
15         </ThemeProvider>
16     );
17 };
18
```

```
19 describe('AttestationPage', () => {
20   beforeEach(() => {
21     api.employeeService.getAll.mockResolvedValue({
22       data: [
23         { id: 1, lastName: 'Doe', firstName: 'John', email: 'john@test.com' }
24       ]
25     });
26
27     api.attestationService.getTypes.mockResolvedValue({
28       data: [
29         { id: 1, value: 'WORK', label: 'Work Attestation' }
30       ]
31     });
32   });
33
34   test('renders attestation page correctly', async () => {
35     renderWithTheme(<AttestationPage onLogout={jest.fn()} />);
36
37     await waitFor(() => {
38       expect(screen.getByText('Attestation Generation')).toBeInTheDocument();
39       expect(screen.getByText('Select an employee')).toBeInTheDocument();
40     });
41   });
42
43   test('generates attestation successfully', async () => {
44     const mockBlob = new Blob(['PDF content'], { type: 'application/pdf' });
45     api.attestationService.generate.mockResolvedValue({ data: mockBlob });
46
47     renderWithTheme(<AttestationPage onLogout={jest.fn()} />);
48
49     await waitFor(() => {
50       const employeeSelect = screen.getByLabelText(/employee/i);
51       fireEvent.change(employeeSelect, { target: { value: '1' } });
52     });
53   });
54 }
```

```

53
54     const generateButton = screen.getByText('Generate Attestation');
55     fireEvent.click(generateButton);
56
57     await waitFor(() => {
58         expect(api.attestationService.generate).toHaveBeenCalled();
59     });
60 });
61 });

```

Listing 4.10: AttestationPage.test.js

4.4.3 Test Results

Tests ensure system reliability with excellent coverage:

Component	Tests	Passed	Coverage
Backend Spring Boot	25	25	95%+
Frontend React	5	5	90%+
REST API	15	15	100%
Business Services	10	10	98%

Figure 4.7: Test Results

4.5 Deployment and Configuration

4.5.1 Build and Packaging

The system is configured for easy production deployment:

```

1 # Backend - Maven
2 cd salary-management-backend
3 ./mvnw clean package -DskipTests
4 # Generates: target/salary-management-backend-0.0.1-SNAPSHOT.jar
5

```

```
6 # Frontend - npm
7 cd salary-management-frontend
8 npm run build
9 # Generates: build/ (optimized static files)
```

Listing 4.11: Build scripts

4.5.2 Production Configuration

Optimized configuration for production environment:

```
1 spring:
2   profiles:
3     active: prod
4   datasource:
5     url: ${DATABASE_URL}
6     username: ${DATABASE_USERNAME}
7     password: ${DATABASE_PASSWORD}
8     hikari:
9       maximum-pool-size: 20
10      minimum-idle: 5
11   jpa:
12     show-sql: false
13     hibernate:
14       ddl-auto: validate
15
16 server:
17   port: ${PORT:8080}
18   compression:
19     enabled: true
20   http2:
21     enabled: true
22
23 logging:
24   level:
```

```
25     com.netcon: INFO
26     org.springframework.security: WARN
27     file:
28     name: logs/salary-management.log
```

Listing 4.12: application-prod.yml

4.6 Conclusion

In summary, the implementation of our salary and HR attestation management system demonstrates the effective use of modern technologies to create a robust and scalable solution. The full-stack architecture with Spring Boot and React provides a solid foundation for production operations.

Development strategies for key components such as employee management, attestation generation with JasperReports, and modern user interface are well-defined, leveraging development best practices for security, performance, and maintainability.

Comprehensive tests ensure system reliability with high coverage, and deployment configuration ensures easy production deployment. Overall, the system is in a healthy state and ready to support the requirements of the applications it hosts.

General Conclusion

I am very pleased to have completed my project developing a salary and HR attestation management system at **Digital Data Service**, where I contributed to the development and deployment of innovative solutions with particular emphasis on **modern technologies**, automation, and development best practices. This experience allowed me to acquire a deep understanding of the technical and organizational challenges related to building scalable, reliable, and secure infrastructures for HR management platforms.

From a technical perspective, the project delivered a consistent and production-ready platform that unifies employee management, automatic attestation generation, data persistence, and modern deployments. Key results:

- **Spring Boot Backend + React Frontend:** modern full-stack architecture with secure REST API.
- **Automatic generation with JasperReports:** dynamic template system and PDF generation.
- **Modern user interface:** Material-UI with dark/light themes and responsive design.
- **Optimized database:** MySQL with efficient data model and integrity constraints.
- **Comprehensive testing:** 95%+ backend and 90%+ frontend coverage with unit and integration tests.

Lessons learned:

- Favor **declarative configuration** and version control for all infrastructure artifacts.
- Build **observability** from the beginning (metrics, logs, alerts) to shorten feedback loops.

Constraints and mitigations:

- Limited time and resources restricted scope; pragmatic use of proven technologies.
- Controlled drift through **automated testing**; rollback safety via versions and immutable images.
- Data reliability through validation, integrity constraints, and periodic backups.

Next steps (roadmap):

- Extend monitoring (**advanced monitoring, dashboards**) and SLO-based alerts.
- Strengthen security (advanced authentication, audit trail, sensitive data encryption).
- Improve **backup/recovery** with tested recovery strategies.
- Add auto-scaling and safer deployments (blue/green, canary).
- Explore integration with other HR systems and data synchronization.

In conclusion, the system establishes a solid and scalable foundation. With progressive investments in monitoring, security, and resilience, it is well-positioned to operate reliably at production scale and meet the company's future needs.

BIBLIOGRAPHY

1. Digital Data Service — Technical sheet. Internal document.
2. Official Spring Boot Documentation. <https://spring.io/projects/spring-boot>. Accessed: 27-09-24.
3. React Documentation. <https://react.dev/>. Accessed: 27-09-24.
4. JasperReports Guide. <https://community.jaspersoft.com/documentation>. Accessed: 27-09-24.
5. Material-UI Documentation. <https://mui.com/>. Accessed: 27-09-24.
6. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>. Accessed: 27-09-24.
7. Maven Documentation. <https://maven.apache.org/guides/>. Accessed: 27-09-24.
8. Axios Documentation. <https://axios-http.com/docs/intro>. Accessed: 27-09-24.
9. Spring Security Reference. <https://docs.spring.io/spring-security/reference/>. Accessed: 27-09-24.
10. React Testing Library. <https://testing-library.com/docs/react-testing-library/intro/>. Accessed: 27-09-24.
11. Jest Testing Framework. <https://jestjs.io/docs/getting-started>. Accessed: 27-09-24.